

arcgisbinding: An R package for integrating R and ArcGIS

Orhun Aydin^{a,b,c,*}, Dmitry Pavlushko^d, Shaun Walbridge^d, Steve Kopp^d, Mark V. Janikas^d

^a Department of Earth and Atmospheric Sciences, Saint Louis University, St. Louis, MO, USA

^b Department of Computer Science, Saint Louis University, St. Louis, MO, USA

^c Taylor Geospatial Institute, St. Louis, MO, USA

^d Environmental Systems Research Institute, Redlands, CA, USA

ARTICLE INFO

Handling Editor: Daniel P Ames

Keywords:

Spatial R

GIS

ArcGIS

Coral bleaching

Ordinal regression

Near-live modeling

ABSTRACT

The open-source geospatial analysis ecosystem in the R language has been growing exponentially, making R a key aspect of statistical analysis in Geographic Information Systems (GIS). In this paper, we present *arcgisbinding*, an open-source R package for R-ArcGIS integration, and its functionality, architecture, and use cases for environmental modeling applications. Integrating R and ArcGIS provides rich end-to-end geospatial analysis workflows and enables a variety of curated spatial and spatiotemporal data in Esri formats for use in the R community. *arcgisbinding* supports geospatial data input and output (I/O), on-the-fly data manipulation, conversion to frequently used spatial R data formats, and interactive data mapping capabilities via *esri.leaflet*. The package allows the creation of R-driven geoprocessing tools, R script tools, that can be executed in ArcGIS. Different integration patterns between R and ArcGIS are elaborated with a case study on modeling coral bleaching severity using near-live data feeds from NOAA's Coral Reef Watch.

1. Introduction

The increase in the volume and variety of spatial and spatiotemporal data is matched with the expansion of spatial analysis ecosystems both as integrated tools in geographic information systems (GIS) software and open-source programming languages (Conrad et al., 2015; Neteler and Mitasova, 2013; QGIS.org, 2023; Scott and Janikas, 2010; Steiniger and Hunter, 2013). In particular, the R language has seen rapid growth in the depth and breadth of its spatial data representations (R. S. Bivand, 2021; Hijmans et al., 2015; Pebesma, 2018, 2022; Pebesma and Bivand, 2012), visualization (Kahle and Wickham, 2015; Karambelkar and Schloerke, 2018), and analysis capabilities (R. Bivand and Nowosad, 2023). In addition to programming languages, open-source GIS software such as QGIS (QGIS.org, 2023) and GRASS (Neteler and Mitasova, 2013), and SAGA (Conrad et al., 2015) contain substantial collection tools for spatial data management, visualization, and analysis. Environmental Systems Research Institute (Esri)'s GIS software and services collection under ArcGIS is frequently used in spatial data production, data management, analysis, and sharing (Ormsby et al., 2012). Other integration packages and bindings have been developed that integrate GIS software with the R language to leverage both ecosystems jointly (R. S. Bivand,

2000; Brenning, 2008; Muenchow et al., 2017). In this paper, we present the R package for binding ArcGIS and the R language, *arcgisbinding*.

The convergence of GIS technologies and open-source programming languages enriches the analysis capabilities within GIS and expands the footprint of the R language in spatial analysis. Additionally, such convergence is essential for addressing challenges in modern geospatial analysis, which often utilizes polyglot technologies (Singleton et al., 2016), as they pertain to reproducibility, replicability, and "recording provenance" (Kedron et al., 2021). Despite the great promise of integrating the rich analysis ecosystem of R with GIS, common interoperability challenges exist for establishing geospatial data pipelines and cross-over functionality. GIS software built with other programming languages, such as Python, generally has breaking changes to R integration once updated. The solution from the open-source community is to deploy a new R package that ensures interoperability between R and the GIS software for predefined versions. For instance, *rgrass*, the family of R packages that enable R integration in GRASS GIS, has a variety of packages such as *rgrass6* and *rgrass7* that operate on specific versions of R and GRASS GIS (R. Bivand et al., 2022). QGIS has taken a similar approach with its R integration. Different versions of R and QGIS are supported by different packages when breaking changes occur, such as

* Corresponding author. Department of Earth and Atmospheric Sciences, Saint Louis University, St. Louis, MO, USA.

E-mail address: orhun.aydin@slu.edu (O. Aydin).

<https://doi.org/10.1016/j.envsoft.2023.105904>

Received 16 November 2023; Accepted 22 November 2023

Available online 5 December 2023

1364-8152/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

RQGIS2 to RQGIS3, which support versions of QGIS with Python 2 and 3 backend, respectively (Muenchow et al., 2017). The challenges related to complex software dependencies are pronounced for GIS and R integration since addition of R increases the volume and dimensions of dependency trees by adding a new language and analysis packages. Another prominent challenge is defining secure and high-fidelity pipelines between web-based GIS and R that ensure security while not limiting access for users. Lastly, a bidirectional integration of R and GIS that allows moving analytical capabilities of one into the other, such as enabling spatial data I/O and GIS analytics in R scripts and integrating R scripts in GIS software, is a necessity for holistic integration.

Open-source community integrations between ArcGIS and R have been limited in scope due to a lack of open-source libraries to access some of Esri's file formats and APIs natively. Nevertheless, open-source R packages that address specific data I/O and geoprocessing-related integration with ArcGIS exist. Regarding geospatial data I/O, *maptools* is designed to address data read and write for Esri ASCII Grid and Open Geospatial Consortium (OGC) standard formats (Roger et al., 2021). The data I/O capability of *maptools* excludes Esri Feature Classes since it requires binding to the private ArcGIS *ArcObjects* binaries. Although there is an open API specification for accessing Esri REST services, there is no comprehensive open-source library for data retrieval. Existing community solutions such as *esri2sf* (Hwang, 2021) for data I/O from Esri's REST APIs involve scraping features without support for image services and are prone to issues pertinent to named user authentication for accessing private geospatial data in online portals. In terms of analysis functionality, packages such as *RpyGeo* (Brenning, 2012) aim to interface with geoprocessing tools in *arcpy* using R's *reticulate* package (Kalinowski et al., 2023). *RpyGeo* runs into the limitations related to dynamic link libraries (DLLs) of ArcGIS Pro, that depending on the ArcGIS Pro and Python version, it cannot import *arcpy*. Recognizing the challenges of these integrations and the value of a robust integration, Esri developed a software bridge uses a mix of proprietary and open libraries and exposes them in an open API.

In this paper, we introduce *arcgisbinding*, an R package for reading, writing, representing, and on the fly manipulating local and cloud-based spatial data. Additionally, we introduce a Conda package *r-arcgis-essentials* encapsulating *arcgisbinding* and frequently used spatial R packages to provide a complete Conda environment to extent ArcGIS Pro's Python distribution. The Conda package addresses versioning issues commonly encountered in polyglot software integration. As of June 2023, the package's GitHub repository (<https://github.com/R-ArcGIS/r-bridge/releases>) had 42947 downloads and the Conda package (<https://anaconda.org/Esri/r-arcgisbinding>) has 2935 downloads.

The core *arcgisbinding* package serves multiple purposes to support different spatial analysis workflows that leverage R and ArcGIS. Firstly, the *arcgisbinding* package performs geospatial data I/O for various local and remote OGC-compliant spatial data sources, including shapefiles, Esri feature classes, Esri rasters, and ArcGIS REST API-based Image and Feature Services. It contains conversion functions representing spatial data with commonly used spatial R objects such as *sf*, *sp*, and *raster**. Secondly, ArcGIS bindings of *arcgisbinding* allow defining R-based script tools, Geoprocessing tools in ArcGIS that run R internally but can be interacted with from the ArcGIS user interface as a Geoprocessing tool. *arcgisbinding*-centric R script tools allow integrating R scripts into larger GIS workflows in frameworks such as the ArcGIS ModelBuilder or serving them in Web GIS as analytic web services accessible via REST interfaces or through the ArcGIS platform. Lastly, *reticulate* integration

makes it possible to make calls to ArcGIS' Python-based spatial data science library, *ArcPy*, for leveraging spatial data science functionality from R. This paper introduces the architecture of *arcgisbinding*, methods for installing it and different ways of working with the package.

2. Package installation and architecture

2.1. Installing *arcgisbinding*

The *arcgisbinding* package can be set up in three different ways.

1. *install.packages* call
2. The Geoprocessing tab in ArcGIS Pro
3. The Conda package *r-arcgis-essentials*

The first and the most commonly used case is using *install.packages*. *arcgisbinding* can be installed from r.esri.com repository with the following call:

```
install.packages("arcgisbinding", repos = "https://r.esri.com", type = "win.binary")
```

At the time of writing, *arcgisbinding* supports R versions ranging from 3.5 to 4.3. *arcgisbinding* For any R version 4.0 and higher, this is the easiest way of setting up the *arcgisbinding* package. For systems where an R version older than 4.0 is being used *arcgisbinding* can be set up using ArcGIS Pro's Geoprocessing tab.

Fig. 1 showcases installing *arcgisbinding* for R version 3.5.3. The UI can be used to install *arcgisbinding* for any R version as the widget searches for all installed versions of R. Note that the Geoprocessing Tab can be used to install *arcgisbinding* for any supported R version.

Lastly, *arcgisbinding* can be set up using Conda. Conda is an open-source package and computational environment management system that facilitates setting up and maintaining environments by creating a self-contained collection of packages that can work harmoniously without version conflicts. Conda has found wide use in both Python and R communities, with the latter providing support for both the language and different IDEs. The *r-arcgis-essentials* Conda package facilitates a computational environment consisting of *arcgisbinding*, *sf*, *raster*, *sp*, *r-mro*, *irkernel*, *leaflet*, and *esri.leaflet*. Installing the package can be done from within the Package Manager interface of Pro, by creating a cloned environment of the *arcgispro-py3* default and using the Add Packages tab to install *r-arcgis-essentials*, or from the command line with:

```
conda create --clone arcgispro-py3 --name pro-r --pinned
conda install r-arcgis-essentials.
```

2.2. Core *arcgisbinding* functionality and architecture

2.2.1. The arc spatial R data frame

ArcObjects are a library of the component object model (COM) written in C++ for efficiently representing geographic data models in ArcGIS. The R package *arcgisbinding* provides bindings into *ArcObjects* for fast and memory-efficient spatial data I/O into R from ArcGIS. The resulting spatial data frame in R is represented as an instance of the *arc.object* class which will be referred to as the arc-type R data frame. This data frame consists of subclasses representing different spatial data types, namely *arc.feature*, *arc.table*, *arc.raster* through direct binding into the *ArcObjects*. The class inheritance for the *arc.object* is depicted in Fig. 2.

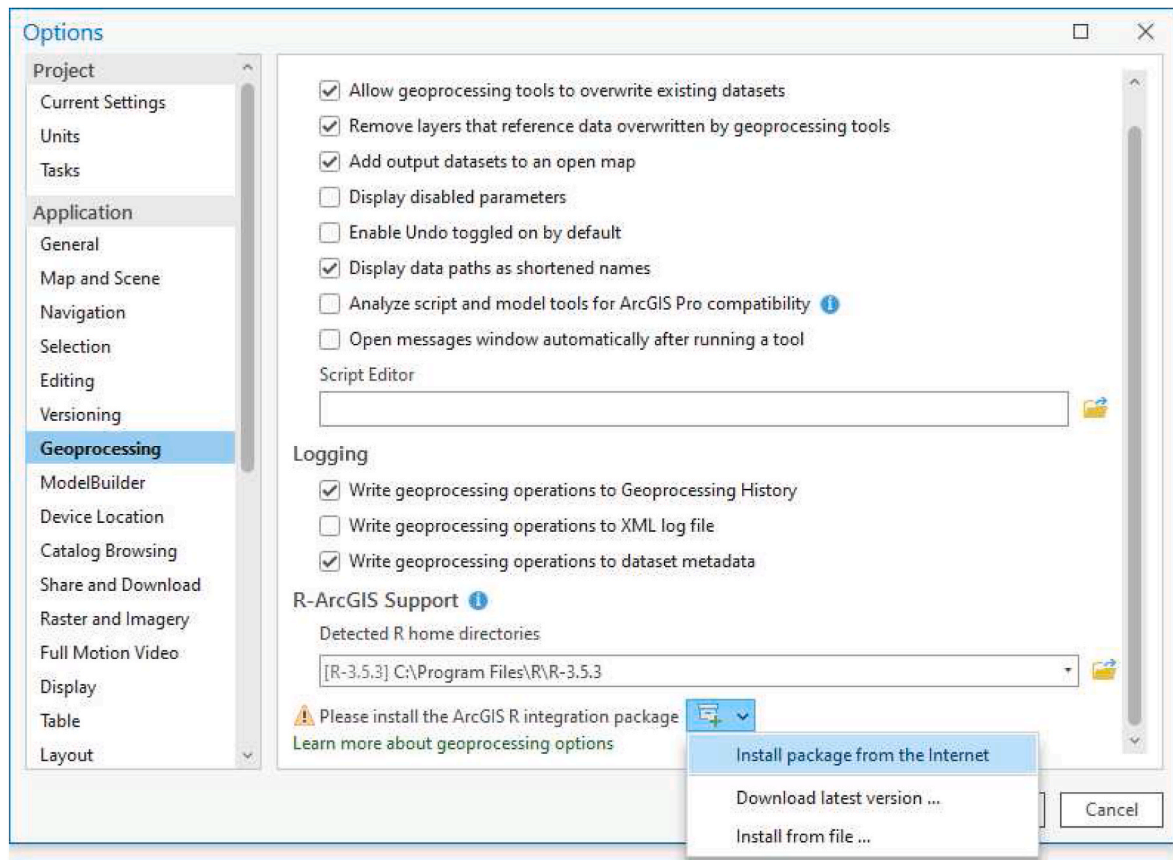


Fig. 1. Geoprocessing Tab UI for installing `arcgisbinding`.

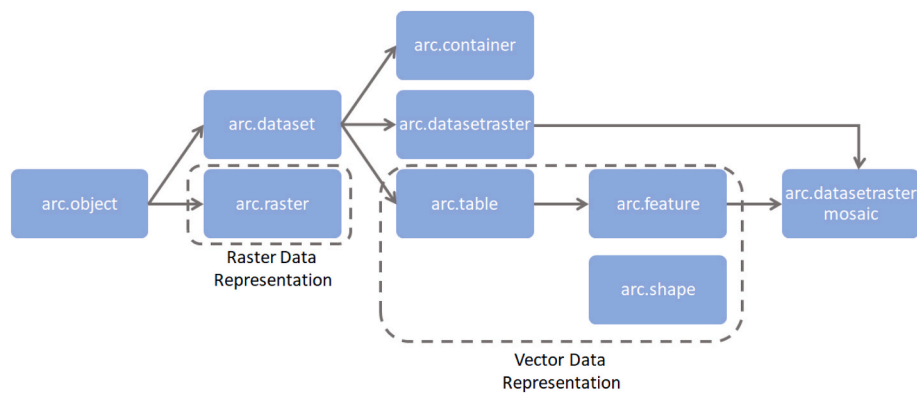


Fig. 2. Class inheritance and spatial data representation inside the `arc` object.

`arc.object` is an R class designed to represent spatial data in a static manner and a dynamic representation for raster objects. Static spatial data representation refers to the representation of spatial data stored on disk or cloud services, such as a shapefile, GeoTIFF, Esri feature class/raster, Esri feature/image service. This paper will refer to this data type as a file dataset.

`arc.dataset` is the main class for static data representation for vector data, raster data, and databases. It represents geospatial data containers: geodatabases and Esri portals, file rasters, and file vector data. One of the children of `arc.dataset`, `arc.container`, represents spatial data containers such as Esri geodatabases or system folders. The two other children of `arc.dataset`, `arc.datasetraster` and `arc.table`, are responsible for representing raster and vector data statically in R. Here, static refers to representing data and metadata on different geospatial data sources as

they exist on disk or cloud storage. `arc.datasetraster` and its child `arc.datasetmosaic`, represent static information on discretization, band values, and spatial reference information from rasters and mosaics, respectively. Vector data requires representing geometry and attribute information separately. `arc.table` is a generic data frame for storing non-spatial attribute information. `arc.feature` extends this class to store shape information pertaining to extent and shape metadata. Lastly, `arc.shape`, a special R list type, stores shape information on every spatial unit in the original dataset. Thus, vector data representations in GIS, such as lines, polygons, and points, `arc.feature` and `arc.table` and `arc.shape` are used to represent metadata, geometry, and attribute information, respectively. `arc.feature` class represents the geometry as an `arc.shape` list, in addition to storing the extent information as an S4 extent object.

The dynamic spatial data representation, `arc.raster` class, is

implemented for raster data which often require substantially large amounts of memory compared to vector data. This class performs on-the-fly manipulation of static raster and mosaic data, represented with *arc.datasetraster* and *arc.datasetrastermosaic*, respectively. *arc.raster* can change the projection, extent, and discretization of incoming raster data stored on disk. On-the-fly manipulation alleviates memory-related bottlenecks while improving the speed of raster I/O. *arc.raster* class defines the *arc.object* for representing rasters in R. The raster file datatypes represented with *arc.datasetraster* and *arc.datasetrastermosaic* are used to represent the data on the static snapshot of the raster in terms of band, extent, cell size, and projection data. *arc.datasetraster* and *arc.datasetrastermosaic* are S4 class objects that store metadata for file rasters and mosaics, respectively. *arc.raster* manipulates these file datatypes to produce an in-memory R raster that can differ from the original raster or mosaic in terms of cell size, extent, and projection.

The *arc.object* is the main representation of spatial data and data containers in R. Depending on the type of dataset being read, such as vector or raster, *arc.object* defines a spatial R data frame referred to as the *arc-type* data frame introduced earlier in this subsection. Note that *arc.object* is the class for representing data, and it requires *arcgisbinding* functions for data I/O, manipulating, and visualization tasks. R language's spatial analysis and visualization infrastructure require specific data types such as *sf*, *sp*, and *raster*. Thus, the spatial data frame represented with an *arc.object* type is not directly useable in many spatial packages in R. *arc.object* is developed as an intermediate data frame that allows fast data I/O to alleviate big data-related bottlenecks in the R ecosystem.

2.2.2. *arcgisbinding* functions

arcgisbinding provides functions for data I/O, on-the-fly spatial data manipulations, user and server authentication, and utility functions for Jupyter R Notebooks and R-based Geoprocessing tool generation in ArcGIS Pro. The following subsections introduce functions that belong to these different capabilities.

2.2.2.1. Data I/O and conversion functions. Data I/O functionality is designed to support all vector, raster, and tabular formats supported by ArcGIS, including all, supported OGC-compliant spatial data sources.

Data I/O entails reading spatial data used in GIS analysis and representing it numerically in R as a spatial R data frame. In addition to local geospatial data such as data in Esri geodatabases, and OGC-compliant local files, *arcgisbinding* can define data I/O pipelines from R into Esri's WebGIS technologies such as Esri's REST API that serve Feature and Image Services.

The package offers many functions for reading data of different spatial formats, converting them to frequently used spatial data types in R as spatial data frames, and exporting them by writing into geodatabases of OGC-compliant data types. Workflow for data I/O and conversion with *arcgisbinding* functions is summarized in Fig. 3, along with functions used for specific data wrangling and representation tasks.

Functions depicted in Fig. 3 are elaborated in the following subsections. Arrows correspond to main spatial data I/O and representation functions in the *arcgisbinding* package. The green circles indicate OGC-compliant spatial data sources and Esri's geospatial cloud data formats such as the Feature Services and Image Services. The dashed area in Fig. 3 indicates spatial data ingestion using the metadata stored in *arc.dataset* format. Note that vector and raster-based geospatial data are read with different functions in *arcgisbinding*.

2.2.2.1.1. *arc.open*. *arc.open* function defines the metadata and attributes summary associated with the spatial data source. Data-read functions require the metadata defined by this function for vector and raster data. Note that *arcgisbinding* is fully connected to Esri's online GIS, ArcGIS Online, and any other cloud offerings derived from it, such as Esri's Living Atlas. Thus, the metadata for any vector or raster data served via an Esri-type REST end-point can be read using *arc.open*, and the data read can be performed with data-read functions.

Metadata summary returned by *arc.open* for vector and raster data are defined in Table 1.

In reading spatial data, *arc.open* is always the first step, as utility functions for reading data require the metadata defined by *arc.open*. *arc.open* reads any vector or raster data type supported in ArcGIS Pro or Desktop.

2.2.2.1.2. *arc.select*. *arc.select* function reads vector data metadata of which is defined by *arc.open*. It is also responsible for reading, spatial subsetting (via a rectangular extent), field-based selection, and generic SQL-based subsetting of vector data. The resulting R data frame is of

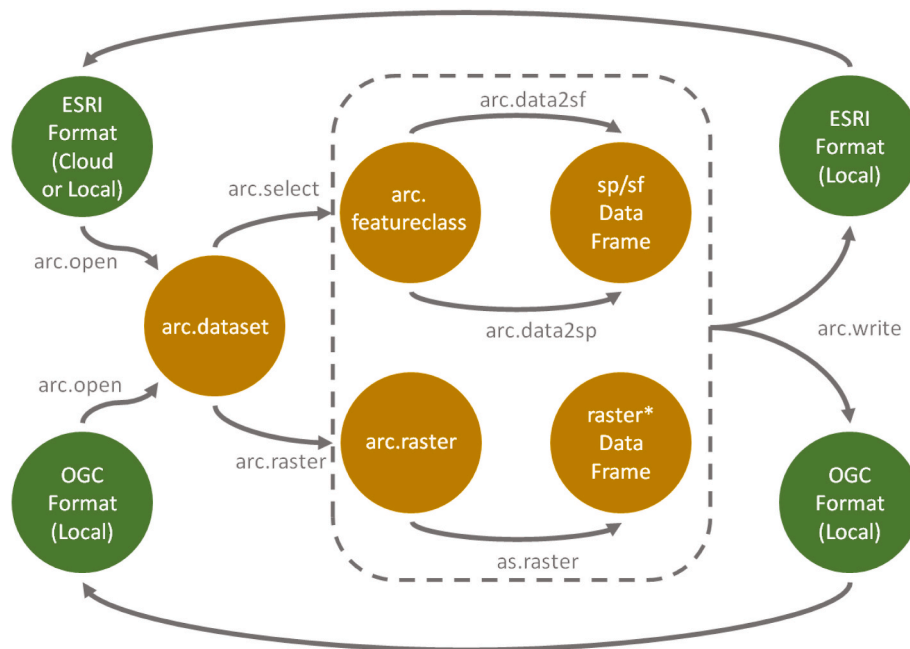


Fig. 3. Spatial data life-cycle in R using *arcgisbinding* functions. Gray arrows indicate *arcgisbinding* functions, green indicate external geospatial data, and yellow are internal R dataframe representations.

Table 1
Vector and Raster Data Attributes for R arc objects.

Vector Data		Raster Data	
Object	Definition	Object	Definition
<i>dataset_type</i>	type of the dataset:	<i>dataset_type</i>	type of the dataset:
<i>path</i>	path to the dataset	<i>format</i>	path to the dataset
<i>fields</i>	fields stored in the attribute table	<i>pixel_type</i>	type of pixel
<i>extent</i>	spatial extent	<i>compression_type</i>	raster compression type
<i>geometry_type</i>	type of geometry	<i>nrow</i>	number of rows
<i>WKT</i>	the well-known text representation of the coordinate reference system	<i>ncol</i>	number of columns
<i>WKID</i>	the well-known ID of the coordinate reference system	<i>extent</i>	spatial extent
		<i>WKT</i>	the well-known text representation of the coordinate reference system
		<i>WKID</i>	the well-known ID of the coordinate reference system
		<i>bands</i>	names of the stored bands
		<i>stats</i>	summary statistics for bands

type *arc.dataset*, where the geometry information is stored in *arc.feature-class* and the attribute field data, is stored in the *arc.table-class*.

arc.select defines on-the-fly data manipulation operations in memory *arc*-objects for local data or using Esri's REST API. Spatial data can be subsetting column-wise with respect to fields required using the *fields* keyword, where *clause* provides SQL queries to subset data row-wise. Lastly, *sr* allows defining the spatial reference for the dataset. If the spatial reference (*sr*) is defined in *arc.select* is different from the one for the original data, data is projected on-the-fly using Esri's projection engine. For vector data, geometry information in *arc.shape-class* is persisted in the spatial data frame defined by *arc.select*.

2.2.2.1.3. *arc.raster*. *arc.raster* function reads raster data from the metadata defined by *arc.open*. The raster read into R as a data frame is an *arc.dataset* type raster is made up of raster band data stored in *arc.datasetraster*. Similarly, the mosaic dataset consists of multiple rasters.

arc.raster can define any Esri-supported raster type, including Image Services, as *arc-type* R rasters. Similar to *arc.select*, it provides functionality for on-the-fly data manipulation, such as extent-based spatial subsets and SQL selections.

2.2.2.1.4. *arc.write*. This data I/O function exports spatial data frames of *arc*, *sp*, *sf*, or raster to any vector or raster data type supported by ArcGIS Pro. Vector data represented as a spatial R data frame in the *arc*, *sp*, or *sf* format can be exported as shapefiles or local feature classes stored in Esri-type geodatabases. Raster data represented with an *arc* or raster type R data frame can also be exported with this function in any raster type that ArcGIS supports.

2.2.2.1.5. *arc.shapeinfo*. *arc-type* R data frame's geometry information can be queried with *arc.shapeinfo*. It returns the type of geometry, the spatial reference (both well-known text (WKT) and well-known ID (WKID)), and whether the data frame contains Z (elevation) or M (point-wise geometry information) values.

2.2.2.1.6. *arc.data2sf*. Simple features (*sf*) is a popular choice for spatial data representation in R. It has gained substantial traction in the spatial R community with more than X downloads at the time this paper is written. For all vector-type *arc-type* data frames, *arc.data2sf* converts the geometry and attribute data stored in the *arc-type* data frame into an *sf* data frame.

2.2.2.1.7. *arc.data2sp*. *Sp* data frames are another commonly used vector data representation in R. *arc.data2sp* function converts an *arc-*

type data frame into an *sp* data frame.

2.2.2.1.8. *as.raster*. Unlike vector data, there is a single modality for raster-type spatial data representation in R, the raster object. The most common spatial R data structure for raster data is the *Raster** object (Hijmans et al., 2015). Raster data read by *arc.raster* can be converted to a *Raster** object with *as.raster*. The *arc-type* raster object stores raster information pertinent to the number of bands, number of pixels, number of columns and rows, spatial extent, and compression type. This conversion function converts the *arc.object* representation to an R raster. The resulting data representation can be used with any R raster function, enabling R packages that work on rasters to data that is read by *arc.raster*.

2.2.2.1.9. *arc.fromP4ToWkt*. This command converts a PROJ.4 coordinate reference system (CRS) string to a well-known text (WKT) representation.

2.2.2.1.10. *arc.fromWktToP4*. This command converts a well-known text (WKT) coordinate reference system (CRS) string to a PROJ.4 representation.

2.2.2.1.11. Authentication functions. *arcgisbinding* requires an ArcGIS Pro or ArcGIS Desktop license to initialize. Thus, it has functions for authentication for activating the R package. Package authentication is an essential step for the function of *arcgisbinding* and needs to be performed whenever the package is loaded to the namespace from scratch.

arcgisbinding also provides functions for authenticating online data stores, particularly ArcGIS Enterprise and ArcGIS Online portals. Functions *arc.check_portal* and *arc.portal_connect* establish connections to password-protected user data in private ArcGIS portals. These functions are a necessary step for accessing private Feature and Image Services.

2.2.2.1.12. *arc.check_product*. *arcgisbinding* requires the ArcObjects, which provide low-level functions for base services such as data access, map analysis, and data representation. Due to this need for ArcObjects, *arcgisbinding* requires a colocated installation of ArcGIS Pro or ArcGIS Desktop containing an active license.

arc.check_product is a utility function that checks for an ArcGIS Pro or Desktop installation after the *arcgisbinding* is imported. The function creates a table output that defines the type of ArcGIS software installed on the machine (ArcGIS Pro or Desktop), the type of license that the ArcGIS software has, and the version of the *arcgisbinding* package. Note that it is essential for *arc.check_product* to return this output, as failure to do will preclude using any of the *arcgisbinding* functions.

2.2.2.1.13. *arc.check_portal*. *arcgisbinding* can read data from cloud-based GIS data sources, particularly ArcGIS REST APIs. For cloud-based data that requires authentication, *arc.check_portal* returns all ArcGIS portals (Online or Enterprise) that the R-instance can authenticate. The active portal instance in ArcGIS Pro is tokenized by *arc.check_portal*, and data can be automatically reached without authentication from the R script. *arc.check_portal* lists all ArcGIS portals listed as inactive in ArcGIS Pro. These portals require an extra authentication step using the *arc.portal_connect* function.

2.2.2.1.14. *arc.portal_connect*. Authenticating ArcGIS portals inactive in ArcGIS Pro from an R instance to access cloud-based GIS data is done with *arc.portal_connect*. This function requires the full path to the remote data portal and user credentials, username, and password to create an on-the-fly token to access cloud-based GIS data. The token is persisted throughout the active R session and is deleted once the session is terminated. This function is used in conjunction with the *arc.check_portal* as only ArcGIS portals defined by this function can be authenticated with *arc.portal_connect*. Thus, *arc.portal_connect* does not authenticate any portal that is not visible in ArcGIS Pro.

It is important to note that this function is not required for publicly available data but only for ArcGIS Portals that require a login.

2.2.2.2. Utility functions. *arcgisbinding* offers functions for integrating R functionality as ArcGIS Pro Geoprocessing tools. Geoprocessing tools are tools with user interfaces in ArcGIS Pro that process geographic data.

The *arc.object* provides direct bindings into ArcObjects for seamless data flow between ArcGIS and R. Thus, when leveraged from a user-designed Geoprocessing tool, R scripts tools can be integrated as ArcGIS Pro Geoprocessing tools. The details of this integration will be introduced under Use Patterns and Case Studies. The functions below are designed as utility tools frequently used when integrating R scripts as Geoprocessing tools. Note that *arc.delete* and *arc.env* have use cases in standalone R scripts and R scripts integrated as tools.

2.2.2.2.1. *arc.delete*. Any local raster, vector data, or arc-type spatial R data frame can be deleted by *arc.delete*. This function can be used with all supported local data to manage the contents of a geodatabase and any spatial data written outside of a geodatabase.

2.2.2.2.2. *arc.env*. The ArcGIS Geoprocessing environment variables can be accessed via the *arc.env* function to ensure that the R Script tool being run from ArcGIS honors environment variables. This function only applies to R script tools that wrap R functionality in ArcGIS, and *arc.env* defines an R data frame that contains information pertinent to geoprocessing environment variables such as the default extent for the output, current workspace, default coordinate system, and default resolution for rasters.

arc.env output can be queried for different geoprocessing environment variables within the R script tool.

2.2.2.2.3. *arc.progress_label*. During the execution of an R script tool in ArcGIS, the progress of the R script can be tracked by using progressors and informative messages provided at the Geoprocessing tool run. *arc.progress_label* defines the informative text to be displayed during the full run to inform the user about the current task being carried out by the R script. This function can be called in multiple parts of the script to give a clear view of the section of the script that is being executed.

2.2.2.2.4. *arc.progress_pos*. *arc.progress_pos* defines a percentage that indicates the progress of the R script tool being executed by ArcGIS. This utility function tracks the process of an R script being executed. A progress label can be associated with a progressor position.

3. Case Study: modeling coral bleaching drivers with ordinal regression on live geospatial data feeds

3.1. Coral reef bleaching

Coral reefs are important for marine biodiversity as they support roughly a quarter of all marine species (Burke et al., 2011; Fisher et al., 2015). The structural complexity of coral reefs promote interdependence between different species, making coral reef health an important aspect to preserve marine biodiversity (ref). Warming oceans and acidification have put substantial stress on coral reefs both in terms of the inhibiting the 3D growth of the CaCO₃ reef framework (Hoegh-Guldberg et al., 2007) and expelling zooxanthellae (phototrophic dinoflagellates algae) from this framework (Douglas, 2003), respectively. The latter process of zooxanthellae leaving the coral reef is also known as coral bleaching due to the coral reef's discoloration following the expulsion of algae (Douglas, 2003).

The increase in frequency of the bleaching events in the last four decades prompted to drivers that are associated with systematically changing environmental conditions, in particular climate change induced stress on coral reefs (Rosenberg and Ben-Haim, 2002). Coral bleaching events are primarily identified with discoloration that can have several underlying causes such as pathogens (Ben-Haim and Rosenberg, 2002), increased Cu and Cd concentrations (Reichelt-Brushett and Harrison, 1999), reduced light penetration at depth (Yentsch et al., 2002). Empirical and numerical modeling studies demonstrated a clear link between anomalously high sea surface temperatures (SSTs) and severity of bleaching events (Brown, 1997; Hoegh-Guldberg, 1999). The space-time patterns of anomalously high temperatures have been studied in-extenso in relation to the severity and extent of bleaching events (Leichter and Miller, 1999; Selig et al., 2010). Similarly, prior work on spatiotemporal variations in bleaching event

severity demonstrates heterogeneity of coral reef response to change in temperature anomalies (Oliver et al., 2009).

As record-breakingly high ocean temperatures are measured almost annually (Cheng et al., 2020, 2021, 2022), and as the amount of near-real time remote sensing data increases (Maxwell et al., 2015), the importance of open-source technologies that can integrate a variety of geospatial data for predictive modeling comes to the fore. In particular, integrative technology that incorporates geographic information systems (GIS), both web-GIS and on-premise GIS, to analysis ecosystems such as R and ArcGIS becomes important for analyzing, modeling, and creating informative data products (Goodchild, 2003).

In this case study, the use of the *arcgisbinding* package is demonstrated to define an ordinal regression model for predicting the severity of coral reef bleaching using near-real time coral bleaching and remotely sensed SST data. The *arcgisbinding* package is demonstrated with three distinct use patterns that implement the same model for predicting coral bleaching severity via ordinal regression implementation from the *ordinal* R package (Christensen, 2018) (ref). The original data sources are in Esri Feature Service format, a Representational state transfer (REST) endpoint hosted by ArcGIS that provides direct access to cloud-based geospatial cloud data including geometry and non-spatial attributes. Feature Services can be public with direct access without any ArcGIS credentials or subscriptions or they can have named-users which requires authentication. NOAA's Coral Reef Watch (CRW) near-live data feed is used for current coral bleaching severity data globally. CRW is used for in-situ report of temperature anomalies and accumulated heat stress. In addition, and the High Resolution Sea Surface Temperature (GHRSSST) compliant level four sea surface temperature (SST) analysis data from Remote Sensing Systems (RSS) on a global 0.1° grid. The RSS SST Esri Image Service (cloud-based geospatial raster) defines a spatially-extensive temperature distribution globally.

3.2. Ordinal regression model of coral bleaching

We model the alert level of coral bleaching as an ordinal variable with explanatory variables that quantify the spatial and temporal distribution of extreme SST. The target variable of our study is the bleaching alert level reported by NOAA's Coral Reef. Coral bleaching alert levels and their definitions are elaborated in Table 2.

We model the impacts of in-situ and proximal temperature anomalies on bleaching severity by combining observed and derived temperature measurements at NOAA CRW stations and remotely sensed SST. The explanatory variables in the regression model are defined in Table 3.

Note that *dhw* and *ssta* variables are in-situ variables observed at NOAA's CRW stations. Satellite imagery-derived average SST ($T_{ave10km}$, $T_{ave20km}$, and $T_{ave30km}$), are quantified for a spatial neighborhood, size of which is defined via a radius. We use *dhw* and *ssta* variables reported by NOAA CRW. Spatially aggregated average SST variables are derived using the RSS SST Esri Image Service and are averaged in circular neighborhoods around NOAA CRW stations.

We use a cumulative link model (Gutiérrez et al., 2016) to the relationships between ordinal alert levels and temperature anomalies at varying spatial lags and temporal indicators of temporal anomalies. The alert level at station i , Y_i , is modelled as a random variable where $Y_i \in \{1, \dots, 4\}$. We do not model the alert level zero. We model the Y_i with the inverse logit link function, f_{logit}^{-1} , where:

Table 2

Description of different alert levels from NOAA's Coral Reef Watch.

Alert Level	Effect
0	Normal conditions
1	Above normal but safe
2	Possible bleaching
3	Significant bleaching likely
4	Severe bleaching and significant mortality likely

Table 3
Description of explanatory variables in modeling the bleaching severity.

Variable Name	Description
<i>dhw</i>	In-situ Degree Heating Week indicates counts the number of days in the past three months, where temperature exceeds max. summertime mean.
<i>ssta</i>	In-situ Sea Surface Temperature (SST) deviation from the average SST from the baseline climatology computed from 1985 to 1990 plus 1993.
<i>T_{ave10km}</i>	Average SST within 10 km radius of the location of interest
<i>T_{ave20km}</i>	Average SST within 20 km radius of the location of interest
<i>T_{ave30km}</i>	Average SST within 30 km radius of the location of interest

$$P(Y_i \leq j) = \gamma_{ij} = f_{\logit}^{-1}(\theta_j - x_i^T \beta) \quad (1)$$

Where j is the ordinal level, θ_j is the intercept in the linear system also known as the threshold value for ordinal level j . The model is a cumulative link model as it defines the probability of a location being under a ordinal level, j , as opposed to being equal to it. $\Theta = [\theta_1, \theta_2, \dots, \theta_k]$ is the monotonically increasing set of all threshold levels where

$$\theta_1 \leq \theta_2 \leq \dots \leq \theta_k \quad (2)$$

The ordinal regression models the impact of explanatory variables on the alert level with a multinomial random variable, $Y_i^* \sim f_{mn}(\pi_i, 1)$, where π_i is the $1 \times k$ array of probabilities for ordinal category membership of location i where $\pi_i = \{P(Y_i = 1), P(Y_i = 2), \dots, P(Y_i = k)\}$.

We model the bleaching alert level Y_i as a linear combination of exogenous drivers as follows:

$$P(Y \leq J) = f_{\logit}^{-1}(\Theta - (\beta_1 dhw + \beta_2 ssta + \beta_3 T_{ave10km} + \beta_4 T_{ave20km} + \beta_5 T_{ave30km})) \quad (3)$$

In Equation (3), J is an ascending array of all alert levels, $J = (1, \dots, k)$.

3.3. Workflows for modeling coral bleaching with ordinal regression

We demonstrate different uses of *arcgisbinding* with three distinct

workflows that showcase working with cloud-based geospatial data, local OGC-compliant data, and integrating R functions into ArcGIS Pro as geoprocessing tools. We name the three workflows: spatial data I/O, R Script Tools in ArcGIS Pro, and *arcpy* and interactive mapping in R Notebooks. The use patterns are not mutually exclusive in terms of their functionality, as they implement the same model on the same data. Rather they showcase uses of cloud-based GIS data and commonly used OGC-compliant geospatial in analysis workflows that include ArcGIS and R. For demonstration purposes, near-live data from CRW and RSS TSS is saved to local OGC-compliant data formats, namely, Esri Feature Class, and Esri Raster to showcase the utility of *arcgisbinding* on frequently used local geospatial data stored in a local geodatabase. The three workflows and their building blocks are depicted in Fig. 4.

In Fig. 4, the cloud data are in the form of ArcGIS API REST endpoints while geodatabase signifies local spatial data that resides in a geospatial database. The data from NOAA's CRW are point localtions (vector data), whereas the ocean surface data served in the Living Atlas by RSS is a cloud-based raster. The color scheme in Fig. 4 indicates the data representation, with gray colors corresponding to spatial data in the *arc*-type R dataframes, and blue corresponding to commonly used spatial data representations such as *sf* and *raster**. The second workflow does not have tabular dataframe representations as it uses *arcgisbinding* to define an ArcGIS Pro tool that integrates the *ordinal* package as a Geoprocessing tool inside ArcGIS Pro.

3.3.1. Spatial data I/O

The spatial data workflow highlights working with cloud-based and local vector and raster data sources to define regression model for bleaching severity. The point-based vector data on coral bleaching is the near-live data from served as a part of NOAA's Coral Reef Watch from an Esri Feature Service. The first step in reading this dataset is defining its geospatial metadata by calling *arc.open* on the feature service URL. As illustrated in Fig. 3, the metadata defined by *arc.open* is used by *arc.select* to read data into R and represent it as an *arc*-type spatial R data frame. The *arc*-type data frame is converted to an *sf* data frame using the *arcgisbinding* conversion function *arc.data2sf*.

Similarly, *arc.open* is used to define the metadata for the SST raster

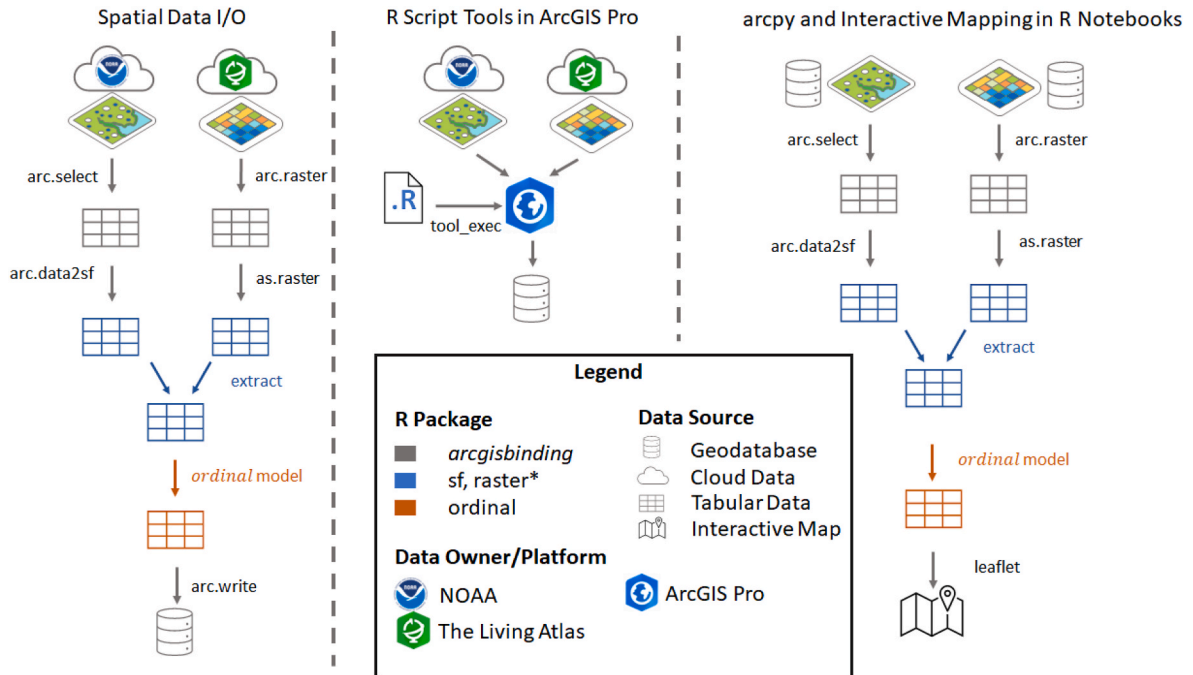


Fig. 4. Three R-ArcGIS integration workflows using *arcgisbinding* for modeling coral bleaching using ArcGIS and R. Each workflow highlights different use patterns of the *arcgisbinding* package to model coral bleaching drivers. Cloud-based geospatial data are symbolized with the authoritative organization's logo.

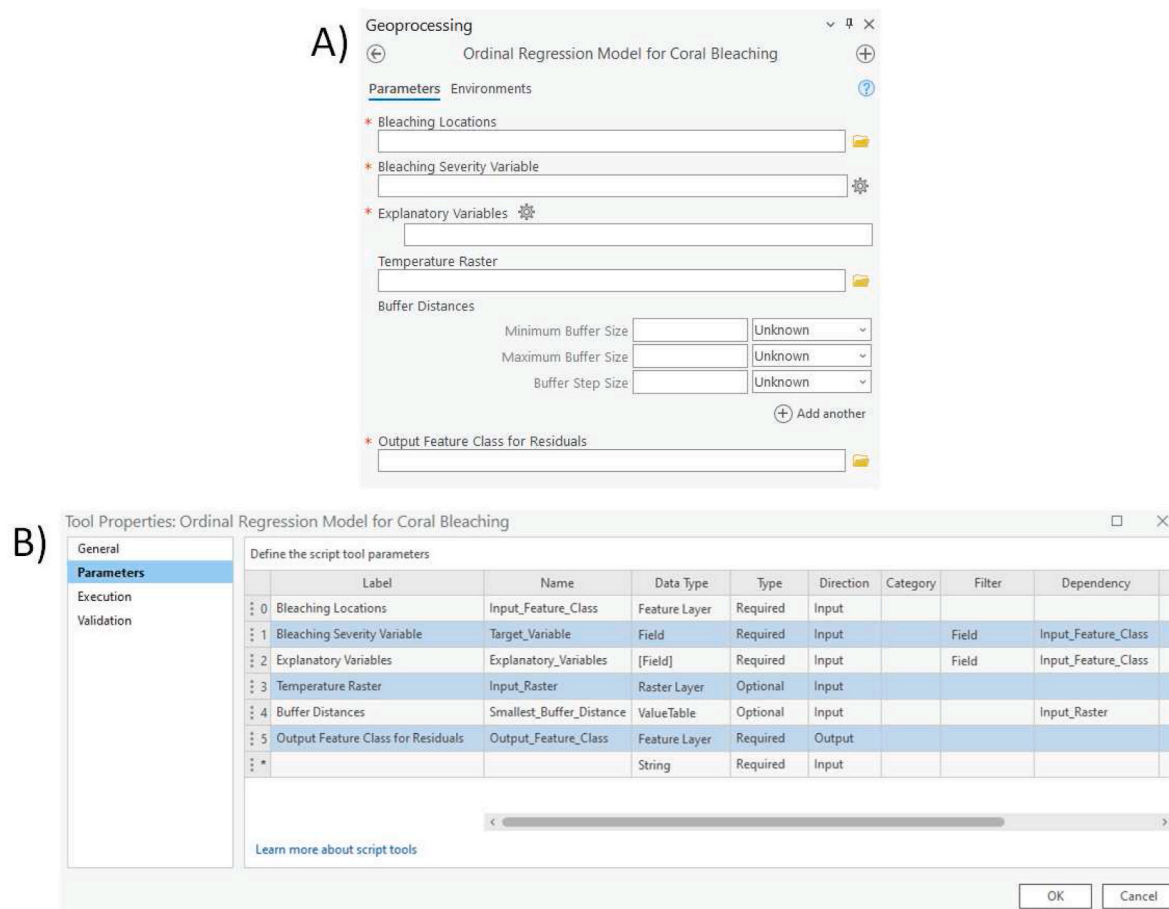


Fig. 5. Tool Properties and User Interface for the Coral Bleaching geoprocessing tool.

served from an Esri Image Service and directly read in as an *arc-type* R raster using *arc.raster*. The *arc-type* raster is converted to an R raster, using the *as.raster* conversion function from the *arcgisbinding* package.

The steps above represent live geospatial data feeds as spatial R data frames, *sf* and *raster*, respectively, bringing both data sources to the spatial R analysis ecosystem. *raster** package's *extract* function is utilized to extract SST values from the Esri Image Service at varying distances to CRW stations and define $T_{ave10km}$, $T_{ave20km}$, and $T_{ave30km}$. The ordinal model

The entire workflow's implementation is provided in the R Notebook (.rmd) included in supplementary materials (https://github.com/orhuna/arcgisbinding_paper).

3.3.2. R script tools in ArcGIS pro

Script tools in ArcGIS allow integrating logic implemented in external libraries to be embedded as ArcGIS Pro Geoprocessing tools. *arcgisbinding* serves as the R integration for script tools and enables R scripts to be executed as geoprocessing tools. Providing the R script as a geoprocessing tool becomes useful to people who know ArcGIS but don't know R.

arcgisbinding requires defining a specific function, *tool_exec*, and can move data from a Geoprocessing tool UI into R seamlessly and return results from the *tool_exec* function as output layers, tool runtime messages, or figures. The general template for R script tools powered by

arcgisbinding is below:

```
tool_exec <- function(in_params, out_params){ }
```

Note that for ArcGIS Pro R script tools, the function and variable names must be used verbatim in the template above. The first input to *tool_exec*, *in_params*, is an R list of lists containing all input parameters from the tool UI. Similarly, *out_params* is an R list of lists containing all output parameters.

An ArcGIS Pro R Script Tool called "Ordinal Regression" that encapsulates the *clmm2* function from the *ordinal* package is defined, and the tool properties and the resulting UI are depicted in Fig. 5.

In the tool properties window in Fig. 5, the ArcGIS Pro script tools have two directions for parameters, input, and output. In this script tool, the input parameters pertain to defining input data and model parameters for the ordinal regression model. The output parameters define file locations for script tool outputs, such as feature classes. For the R function *tool_exec*, *in_params* and *out_params* variables contain UI parameters labeled input and output in their direction definition, respectively. Lastly, the ArcGIS Pro R script tool and the R script containing the *tool_exec* function are connected by assigning the R script to be the source of the script tool under the Execution tab of the tool properties window displayed in Fig. 5. The geoprocessing tool allows integrating the R script tool into Pythonic spatial analysis workflows as it can be called as a Python toolbox through the *arcpy* package with the code snippet below.

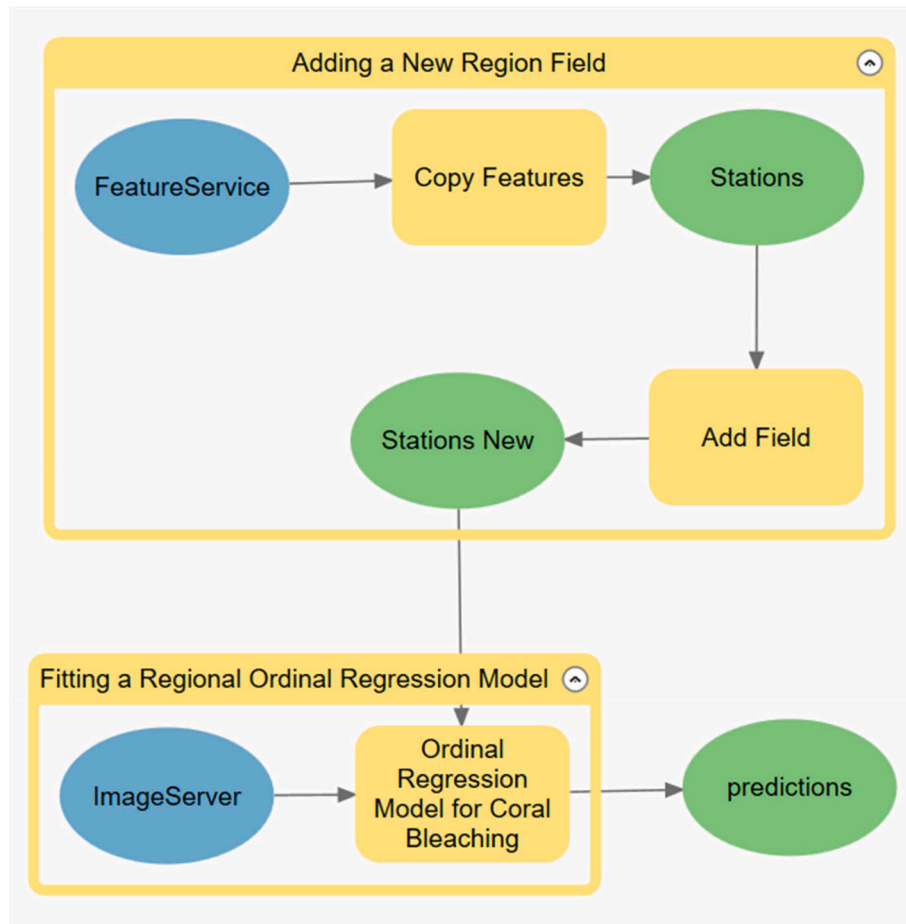


Fig. 6. Sample ArcGIS ModelBuilder containing that integrates the R Script Tool with other geoprocessing tools in a workflow.

```

arcpy.ImportToolbox(r" sample_toolbox.atbx")
arcpy.sampletoolbox.OrdinalRegression(
    Input_Feature_Class=in_fc,
    Target_Variable="alert",
    Explanatory_Variables="dhw;ssta",
    Input_Raster= in_raster,
    Smallest_Buffer_Distance="'10 Kilometers' '30 Kilometers' '10 Kilometers'",
    Output_Feature_Class=None)
  
```

The script tool above and the associated R script executes the same workflow discussed in the previous subsection related to spatial data I/O. The R script tool's execution resides fully inside threads occupied by ArcGIS Pro. Thus, R script tools created using *arcgisbinding* can be scheduled for execution or integrated into ArcGIS Model Builder workflows like other Geoprocessing tools. An example Model Builder workflow containing the R script tool is showcased below.

Fig. 6 depicts an ArcGIS Model Builder workflow that executes a data

Table 4

Examples of translating *arcpy* calls from Python to R.

arcpy Python Command	R Equivalent
<code>arcpy.sa.EucDistance(input_fc, None, cell_size, None, "PLANAR", None, None)</code>	<code>arcpy\$sa\$EucDistance(input_fc, NULL, cell_size, NULL, "PLANAR", NULL, NULL)</code>
<code>arcpy.management.ProjectRaster(input_raster, output_raster, output_cs, None, None, input_cs, "NO_VERTICAL")</code>	<code>arcpy\$management\$ProjectRaster(input_raster, output_raster, output_cs, NULL, NULL, input_cs, "NO_VERTICAL")</code>

wrangling step using ArcGIS Pro's Copy Features and Add Field tools before defining the ordinal regression model in the R script tool "Ordinal Regression Model for Coral Bleaching." Fig. 6 shows the flexibility of.

3.3.3. Integrating *arcpy* in R scripts and interactive mapping

This example showcases another using pattern for the *arcgisbinding* package for integrating ArcGIS Pro Geoprocessing functionality in R scripts. Additionally, *esri.leaflet*, an R package with bindings to the *leaflet* package, is added to the workflow to showcase interactive mapping. Note that *esri.leaflet* and *arcgisbinding* are separate R packages without interdependencies. The workflow presented in Fig. 6 is implemented in an R Notebook (.Rmd) for ease of reading and reproducibility.

The current *arcpy* integration is performed via *reticulate*, an R package for calling Python functions. In particular, the *reticulate* function *use_python* can bind to a specific Python interpreter if multiple installations are present. Note that *arcpy* integration only works for Python interpreters with *arcpy* installed and ArcGIS licenses activated.

The example script implements the workflow depicted in Fig. 6 by calling Euclidean Distance, Project Raster, and Convert Time Field Geoprocessing tools from the R instance. The first step in integrating

Geoprocessing tools is importing the `arcpy` package as follows:

```
arcpy <- import("arcpy")
```

`arcpy` functions can be called using R-specific syntax, examples of translating `arcpy` calls from Python to R are elaborated in Table 4.

Note that `arcpy` calls in Table 4 define outputs, such as `output_raster`, in the hard drive. `Arcgisbinding` becomes particularly useful after `arcpy` calls as data read functionality can be used to represent all function outputs as spatial R data frames (`sf` or `raster*`) for further analysis in R. Some functions, such as the Euclidean Distance tool, require extensions, such as the Spatial Analyst Extension. In these cases, `arcpy` commands like the one below can be used from an R script to check out the extension to enable making the call.

```
Arcpy$CheckOutExtension("Spatial")
```

Terminating the R kernel automatically returns all checked-out `arcpy` extensions.

`r-arcgis-essentials` Conda package is useful in use patterns that involve *reticulate* integration. `Arcpy` integration through *reticulate* adds the Python version as a new dimension in the dependency matrix. Since Conda sets up prescribed versions of all software and packages, the chance of version conflicts between R, Python, and `arcgisbinding` is minimized. This subsection only introduces the novel aspects of the `arcpy` integration in R, and the entire R notebook containing the full implementation is provided separately in the repository.

3.4. Case Study results

We visually explore the coral bleaching patterns using the interactive mapping unctiionrelationships between predictors and the bleaching alert levels using a Bepplot.

Regression coefficients of the ordinal regression model with spatial lags indicate that lagged temperature variable coefficients are insignificant in the model. Conversely, the sea surface temperature anomaly (*ssta*) and degree heating weeks (*dhw*) have significant contributions to the coral bleaching severity. The second model where the SST within a 10 km radius (*T_ave_10.km*) is included along with *ssta* and *dhw*, the significance of the regression coefficient for *T_ave_10.km* increases. The increase in the *T_ave_10.km* coefficient significance is due to the lack of multicollinearity for the model with one spatial lag. We further compare the performance of the two models with a likelihood ratio test between to assess the efficacy of including multiple spatial lags of SSTs in predicting coral bleaching severity. The ANOVA test for the likelihood ratios of the two models indicate that the differences in likelihoods, likelihood ratio of 0.5108, is not statistically significant, *p-value* = 0.7746. The thresholds for bleaching severity for the two models are elaborated in Table 5.

Note that the thresholds values in Table 6 are similar for both models

Table 5

Coefficients and their statistical significance for the ordinal regression model with and without spatial SST lags.

Variable (X)	Model with Multiple Spatial Lags		Model One Spatial Lag	
	Coefficient (β)	P-value	Coefficient (β)	P-value
<i>T_ave_10.km</i>	-4.4150	0.4772	-0.2154	0.147
<i>T_ave_20.km</i>	8.8959	0.4825	NA	NA
<i>T_ave_30.km</i>	-4.7177	0.5006	NA	NA
<i>ssta</i>	0.9117	0.1161	0.8647	0.122
<i>dhw</i>	1.1591	5.5917 e-8	1.1706	5.96 e-8

Table 6

Comparison of bleaching severity thresholds for the two models.

Thresholds (Θ)	Model with Multiple Spatial Lags	Model One Spatial Lag
1 2	2.3339	2.7115
2 3	4.0161	4.3537
3 4	6.9010	7.2154

with both models indicating a larger increase in the threshold to switch from significant bleaching, $\theta_{2|3}$, to severe bleaching, $\theta_{3|4}$, with mortality.

4. Conclusions

In this paper, we presented *arcgisbinding*, an R package for integrating ArcGIS Pro and R. The package provides spatial data I/O for various feature, raster, and tabular types supported by ArcGIS and OGC-supported formats. The package supports reading local and cloud-based geospatial data sources, Esri's Feature, Image Services, and OGC Features. *arcgisbinding* provides conversion functions for spatial data representation in commonly spatial R data types, namely `sf` and `raster*`. In addition to spatial data representation and wrangling, *arcgisbinding* offers functionality for accessing online data portals through its tokenization capability that allows reaching geospatial data accessible to named users. Lastly, the package provides utility functions for R script tool creation for encapsulating R functions in user-created ArcGIS Geoprocessing tools.

The paper showcases three different use patterns of *arcgisbinding*. The use cases are not meant to be mutually exclusive; different aspects of each use pattern can be combined in different geocomputation workflows. The three use cases presented are the most distinct and recurring among the tens of thousands of *arcgisbinding* users. An R package *ordinal* is used to showcase the use of *arcgisbinding* in defining an exploratory regression models for coral bleaching severity using in-situ and remotely sensed observations. The ordinal regression model implemented in R exemplifies a common motivation for *arcgisbinding*, accessing R analysis functions not available in ArcGIS to leverage both analysis ecosystems jointly. The first use pattern showcases using *arcgisbinding* for spatial data I/O for R-centric analysis, a Feature Service, and an Esri raster are read as spatial R data frames for analysis in an R script. The second use pattern showcases the creation of R script tools, ArcGIS Pro Geoprocessing tools, with analysis performed with R. Lastly, an R notebook use pattern showcases the use of *arcgisbinding* alongside *arcpy* and *esri.leaflet* for integrating ArcGIS Geoprocessing tools and interactive mapping in R scripts. Coral bleaching severity analysis indicates the value of integrating geospatial data networks to the rich analysis ecosystems of R and ArcGIS.

Funding sources

Orhun Aydin acknowledges the Taylor Geospatial Institute (TGI) grant, PROJ-000323. All authors acknowledge the Spatial Statistics and R-ArcGIS Bridge team at Esri in funding human hours spent in developing the *arcgisbinding* package.

Software and data availability section

Arcgisbinding package is available in the following repo, <https://github.com/R-ArcGIS/r-bridge>.

Software and data are available in the following repo, https://github.com/orhuna/arcgisbinding_paper.

Declaration of competing interest

Orhun Aydin reports financial support was provided by Esri Inc. Dmitry Pavlushko reports financial support was provided by Esri Inc. Shaun Walbridge reports financial support was provided by Esri Inc. Steve Kopp reports financial support was provided by Esri Inc. Mark V. Janikas reports financial support was provided by Esri Inc.

Data availability

Data will be made available on request.

References

- Ben-Haim, Y., Rosenberg, E., 2002. A novel *Vibrio* sp. pathogen of the coral *Pocillopora damicornis*. *Marine Biology* 141 (1), 47–55. <https://doi.org/10.1007/s00227-002-0797-6>.
- Bivand, R., Krug, R., Neteler, M., Jeworutzki, S., & Vanderhaeghe, F. (2022). Package “Rgrass7”.
- Bivand, R., Nowosad, J., 2023. CRAN task view: analysis of spatial data. <https://cran.r-project.org/view=Spatial>.
- Bivand, R.S., 2000. Using the R statistical data analysis language on GRASS 5.0 GIS database files. *Comput. Geosci.* 26 (9–10), 1043–1052. [https://doi.org/10.1016/S0098-3004\(00\)00057-1](https://doi.org/10.1016/S0098-3004(00)00057-1).
- Bivand, R.S., 2021. Progress in the R ecosystem for representing and handling spatial data. In *Journal of Geographical Systems*. Springer Berlin Heidelberg 23 (Issue 4). <https://doi.org/10.1007/s10109-020-00336-0>.
- Brenning, A., 2008. Statistical geocomputing combining R and SAGA: the example of landslide susceptibility analysis with generalized additive models. In: Böhne, J., Blaschke, T., Montanarella, L. (Eds.), *SAGA – Seconds Out (Hamburger Beiträge zur Physischen Geographie und Landschaftsökologie)*, vol. 19, pp. 23–32.
- Brenning, A., 2012. RPyGeo: ArcGIS Geoprocessing in R via Python. R package version 0.9-3, pp. 410–423. <https://CRAN.R-project.org/package=RPyGeo>.
- Brown, B.E., 1997. Coral bleaching: causes and consequences. *Coral Reefs* 16 (Suppl. 1), 129–138. <https://doi.org/10.1007/s003380050249>.
- Burke, L., Reyter, K., Spalding, M., Perry, A., 2011. Reefs at risk revisited. *Defenders* 74 (3). <http://www.pubmedcentral.nih.gov/articlerender.fcgi?artid=3150666&tool=pmcentrez&rendertype=abstract>.
- Cheng, L., Abraham, J., Trenberth, K.E., Fasullo, J., Boyer, T., Locarnini, R., Zhang, B., Yu, F., Wan, L., Chen, X., Song, X., Liu, Y., Mann, M.E., Reseghetti, F., Simoncelli, S., Gouretski, V., Chen, G., Mishonov, A., Reagan, J., Zhu, J., 2021. Upper Ocean temperatures hit record high in 2020. *Adv. Atmos. Sci.* 38 (4), 523–530. <https://doi.org/10.1007/s00376-021-0447-x>.
- Cheng, L., Abraham, J., Trenberth, K.E., Fasullo, J., Boyer, T., Mann, M.E., Zhu, J., Wang, F., Locarnini, R., Li, Y., Zhang, B., Tan, Z., Yu, F., Wan, L., Chen, X., Song, X., Liu, Y., Reseghetti, F., Simoncelli, S., et al., 2022. Another record: ocean warming continues through 2021 despite La niña conditions. *Adv. Atmos. Sci.* 39 (3), 373–385. <https://doi.org/10.1007/s00376-022-1461-3>.
- Cheng, L., Abraham, J., Zhu, J., Trenberth, K.E., Fasullo, J., Boyer, T., Locarnini, R., Zhang, B., Yu, F., Wan, L., Chen, X., Song, X., Liu, Y., Mann, M.E., 2020. Record-setting ocean warmth continued in 2019. *Adv. Atmos. Sci.* 37 (2), 137–142. <https://doi.org/10.1007/s00376-020-9283-7>.
- Christensen, R.H.B., 2018. Cumulative Link Models for Ordinal Regression with the R Package Ordinal. Unsubmitted Manuscript, Christensen 2019, pp. 1–40 cran.uni-muenster.de/web/packages/ordinal/vignettes/clm_article.pdf.
- Conrad, O., Bechtel, B., Bock, M., Dietrich, H., Fischer, E., Gerlitz, L., Wehberg, J., Wichmann, V., Böhner, J., 2015. System for automated geoscientific analyses (SAGA) v. 2.1.4. *Geosci. Model Dev. (GMD)* 8 (7), 2271–2312. <https://doi.org/10.5194/gmd-8-1991-2015>.
- Douglas, A.E., 2003. Coral bleaching - how and why? *Mar. Pollut. Bull.* 46 (4), 385–392. [https://doi.org/10.1016/S0025-326X\(03\)00037-7](https://doi.org/10.1016/S0025-326X(03)00037-7).
- Fisher, R., O’Leary, R.A., Low-Choy, S., Mengersen, K., Knowlton, N., Brainard, R.E., Caley, M.J., 2015. Species richness on coral reefs and the pursuit of convergent global estimates. *Curr. Biol.* 25 (4), 500–505. <https://doi.org/10.1016/j.cub.2014.12.022>.
- Goodchild, M.F., 2003. Geographic information science and systems for environmental management. *Annu. Rev. Environ. Resour.* 28, 493–519. <https://doi.org/10.1146/annurev.energy.28.050302.105521>.
- Gutiérrez, P.A., Pérez-Ortiz, M., Sánchez-Monederó, J., Fernández-Navarro, F., Hervás-Martínez, C., 2016. Ordinal regression methods: survey and experimental study. *IEEE Trans. Knowl. Data Eng.* 28 (1), 127–146. <https://doi.org/10.1109/TKDE.2015.2457911>.
- Hijmans, R., Etten, J. van, Sumner, M., Cheng, J., Baston, D., Bevan, A., Bivand, R., Busetto, L., Canty, M., Fasoli, B., Forrest, D., Ghosh, A., Golicher, D., Gray, J., Greenberg, J. A., Hiemstra, P., Hingee, K., Ilich, A., Geosciences, I. for M. A., et al. (2015). Package “raster.” R Package, 734..
- Hoegh-Guldberg, O., 1999. Climate change, coral bleaching and the future of the world’s coral reefs. *Mar. Freshw. Res.* 50 (8), 839–866. <https://doi.org/10.1071/MF99078>.
- Hoegh-Guldberg, O., Mumby, P., Hooten, A., Steneck, R., Greenfield, P., Gomez, E., Harvell, C., Sale, P., Edwards, A., Caldeira, K., Knowlton, N., Eakin, C., Iglesias-Prieto, R., Muthiga, N., Bradbury, R., Dubi, A., Hatziolos, M., 2007. Coral Reefs under Rapid Climate Change and Ocean Acidification, vol. 318, pp. 1737–1742. <https://doi.org/10.1126/science.1152509>. December 2007.
- Hwang, Y., 2021. esri2sf. <https://github.com/yonghah/esri2sf>.
- Kahle, D., Wickham, H., 2015. ggmap: spatial Visualization with ggplot2. *R Journal* 5 (1), 144–161. <https://doi.org/10.1051/ps/2015001>.
- Kalinowski, T., Ushey, K., Allaire, J., 2023. Package “reticulate” 1.27, 1–49. <https://cran.r-project.org/web/packages/reticulate/reticulate.pdf>.
- Karambelkar, B., Schloerke, B., 2018. Package “leaflet.esri.” <https://cran.r-project.org/package=leaflet.esri>.
- Kedron, P., Li, W., Fotheringham, S., Goodchild, M., 2021. Reproducibility and replicability: opportunities and challenges for geospatial research. *Int. J. Geogr. Inf. Sci.* 35 (3), 427–445. <https://doi.org/10.1080/13658816.2020.1802032>.
- Leichter, J.J., Miller, S.L., 1999. Predicting high-frequency upwelling: spatial and temporal patterns of temperature anomalies on a Florida coral reef. *Contin. Shelf Res.* 19 (7), 911–928. [https://doi.org/10.1016/S0278-4343\(99\)00004-7](https://doi.org/10.1016/S0278-4343(99)00004-7).
- Maxwell, S.M., Hazen, E.L., Lewison, R.L., Dunn, D.C., Bailey, H., Bograd, S.J., Briscoe, D.K., Fossette, S., Hobday, A.J., Bennett, M., Benson, S., Caldwell, M.R., Costa, D.P., Dewar, H., Eguchi, T., Hazen, L., Kohin, S., Sippel, T., Crowder, L.B., 2015. Dynamic ocean management: defining and conceptualizing real-time management of the ocean. *Mar. Pol.* 58, 42–50. <https://doi.org/10.1016/j.marpol.2015.03.014>.
- Muenchow, J., Schratz, P., Brenning, A., 2017. RQGIS: integrating R with QGIS for statistical geocomputing. *R Journal* 9 (2), 409–428. <https://doi.org/10.32614/rj-2017-067>.
- Neteler, M., Mitsova, H., 2013. Open Source GIS: a GRASS GIS Approach. *Pringer Science & Business Media*, p. 689.
- Oliver, J.K., Berkelmans, R., Eakin, C.M., 2009. Coral Bleaching in Space and Time, vols. 21–39. https://doi.org/10.1007/978-3-540-69775-6_3.
- Ormsby, T., Napoleon, E., Burke, R., Groessl, C., Feaster, L., 2012. Getting to Know ArcGIS Desktop: Basics of ArcView, ArcEditor, and ArcInfo. ESRI, Inc. <https://doi.org/10.5642/passwrd.20121301.19>.
- Pebesma, E., 2018. Simple features for R: standardized support for spatial vector data. *R Journal* 10 (1), 439–446. <https://doi.org/10.32614/rj-2018-009>.
- Pebesma, E., 2022. Stars: spatiotemporal arrays, raster and vector data cubes. In: *Repository CRAN*, pp. 1–41. <https://cran.r-project.org/package=stars>.
- Pebesma, E., Bivand, R., 2012. Package “sp.” The Comprehensive R Archive Network. QGIS.org, 2023. QGIS Geographic Information System. QGIS Association.
- Reichert-Brushett, A.J., Harrison, P.L., 1999. The effect of copper, zinc and cadmium on fertilization success of gametes from scleractinian reef corals. *Mar. Pollut. Bull.* 38 (3), 182–187. [https://doi.org/10.1016/S0025-326X\(98\)00183-0](https://doi.org/10.1016/S0025-326X(98)00183-0).
- Roger, B., Archer, E., Baddeley, A., Bearman, N., Forrest, D., Friendly, M., Stokely, M., Turner, R., Rogerbivandnhhno, M.R.B., 2021. Package “maptools” R topics documented : Rosenberg, E., & Ben-Haim, Y. (2002). Microbial diseases of corals and global warming. *Environ. Microbiol.* 4 (6), 318–326. <https://doi.org/10.1046/j.1462-2920.2002.00302.x>.
- Rosenberg, E., Ben-Haim, Y., 2002. Microbial diseases of corals and global warming. *Environ. Microbiol.* 4 (6), 318–326.
- Scott, L., Janikas, M.V., 2010. Spatial statistics in ArcGIS. In: *Handbook of Applied Spatial Analysis*. Springer Berlin Heidelberg, pp. 27–41.
- Selig, E.R., Casey, K.S., Bruno, J.F., 2010. New insights into global patterns of ocean temperature anomalies: Implications for coral reef health and management. *Global Ecol. Biogeogr.* 19 (3), 397–411. <https://doi.org/10.1111/j.1466-8238.2009.00522.x>.
- Singleton, A.D., Spielman, S., Brunsdon, C., 2016. Establishing a framework for open geographic information science. *Int. J. Geogr. Inf. Sci.* 30 (8), 1507–1521. <https://doi.org/10.1080/13658816.2015.1137579>.
- Steiniger, S., Hunter, A.J.S., 2013. The 2012 free and open source GIS software map - a guide to facilitate research, development, and adoption. *Comput. Environ. Urban Syst.* 39, 136–150. <https://doi.org/10.1016/j.compenvurbysys.2012.10.003>.
- Yentsch, C.S., Yentsch, C.M., Cullen, J.J., Lapointe, B., Phinney, D.A., Yentsch, S.W., 2002. Sunlight and water transparency: cornerstones in coral research. *J. Exp. Mar. Biol. Ecol.* 268 (2), 171–183. [https://doi.org/10.1016/S0022-0981\(01\)00379-3](https://doi.org/10.1016/S0022-0981(01)00379-3).